Student Mental Health

Monitoring System

SE LAB Experiment-07 BATCH-D

Requirements Analysis Document

Submitted By:

Mokshi Shah
241071068

Vanshika Shah
241071069

Experiment Objectives

1. Modeling Interactions among Objects with CRC Cards
2. Identifying Associations
3. Identifying Aggregates
4. Identifying Attributes
5. Model State-Dependent Behavior of Individual Objects
6. Modeling Inheritance Relationships

April 11, 2026

Contents

Contents

1 MODELING INTERACTIONS WITH CRC CARDS

What is a CRC Card? A **C**lass–**R**esponsibility–**C**ollaborator (CRC) card models each analysis class by listing: (1) what it is responsible for knowing or doing, and (2) which other classes it must collaborate with to fulfil those responsibilities. CRC cards are derived directly from the use-case analysis classes identified in Experiment 04A.

1.1 Entity Classes

1.1.1 Class: User

Class: User [*Abstract Entity*]	
Responsibilities	**Collaborators**
Know userId, email, role, locked status	AuthController
Authenticate user credentials	SessionManagementController
Track failed login attempts	LoginUI
Support role-based access enforcement	RoleAssignmentController
Know account creation and last-login timestamps	AuditLog

1.1.2 Class: ScreeningRecord

Class: ScreeningRecord [*Entity*]	
Responsibilities	**Collaborators**
Know PHQ-9, GAD-7, and PSS scores for a student	ScreeningController
Know submission timestamp and version number	ScreeningUI
Know the derived risk tier at time of submission	RiskProfile
Persist validated screening responses	DataCollectionController
Support retrieval of historical screening history	StatusController

1.1.3 Class: MoodLog

Class: MoodLog [*Entity*]	
Responsibilities	**Collaborators**
Know daily mood score (1–10), sleep hours, stressors	MoodController
Know log date and associated studentId	CheckInUI
Feed stored data into continuous risk algorithm	AnalyticsController
Support partial-save for incomplete entries	RiskProfile

1.1.4 Class: RiskProfile

Class: RiskProfile [*Entity*]	
Responsibilities	**Collaborators**
Know current composite risk score and risk tier	RiskController
Know trend history (JSON series of past scores)	AnalyticsController
Know last classification timestamp	StatusController
Trigger counsellor alert when tier reaches High	NotifController
Support retrieval of latest profile per student	PatternReport

1.1.5 Class: CounselingSession

Class: CounselingSession [*Entity*]	
Responsibilities	**Collaborators**
Know session date, time, type (in-person / virtual), status	BookingController
Know studentId and counsellorId for the appointment	CounselingScheduleInterface
Know session notes (encrypted, post-session)	NotifController
Record cancellation reason and timestamp if cancelled	AuditLog
Trigger 24-hour automated reminders to both parties	SessionDB

1.1.6 Class: NotificationLog

Class: NotificationLog [*Entity*]	
Responsibilities	**Collaborators**
Know notification type, recipient userId, delivery channel	NotifController
Know delivery timestamp and acknowledgement status	NotificationUI
Support retry logic for failed email deliveries	EmailService
Log frequency to enforce notification capping	System / Analytics Engine

1.1.7 Class: PatternReport

Class: PatternReport [*Entity*]	
Responsibilities	**Collaborators**
Know studentId and anomalies detected (JSON)	AnalyticsController
Know generation timestamp and data-sufficiency flag	MLModelInterface
Pass structured output to risk detection (UC-10)	RiskController
Flag students with insufficient data for manual review	CounselorDashboardUI

1.1.8 Class: ConsentRecord

Class: ConsentRecord [*Entity*]	
Responsibilities	**Collaborators**
Know studentId, guardianId, consent status, grant date	PrivacyController
Support revocation of consent by the student	SummaryUI
Provide gate-check before any guardian data access	AuditLog
Record consent change events with timestamp	ConsentDB

1.1.9 Class: StudentAcadData

Class: StudentAcadData [*Entity*]	
Responsibilities	**Collaborators**
Know grades, attendance records, and campus engagement metrics	DataCollectController
Know SIS data collection timestamp and sync version	SISApiGateway
Support schema validation before storage	AnalyticsController
Feed normalised data into pattern detection models	MLModelInterface

1.1.10 Class: AuditLog

Class: AuditLog [*Entity*]	
Responsibilities	**Collaborators**
Know actor identity, action type, target object, timestamp	AccessControlController
Ensure tamper-evidence for all recorded entries	PrivacyComplianceController
Support query by actor, date range, and action type	SecurityConfigurationInterface
Persist all configuration change events	AdminPanelUI

1.1.11 Class: DashboardReport

Class: DashboardReport [*Entity*]	
Responsibilities	**Collaborators**
Know reporting period, scope (admin/counsellor), and format	TrendAnalysisController
Know aggregated anonymised KPI metrics and trend charts	AdminDashboardUI
Support export in PDF and CSV formats	DashboardController
Ensure no student-identifiable fields in admin-level exports	AccessControlController

1.2 Control Classes

1.2.1 Class: AuthController

Class: AuthController [*Control*]	
Responsibilities	**Collaborators**
Validate email and password against stored credentials	User
Orchestrate 2FA verification flow	LoginUI
Lock account after 5 consecutive failed attempts	SessionManagementController
Issue session token on successful authentication	AuditLog

1.2.2 Class: RiskController

Class: RiskController [*Control*]	
Responsibilities	**Collaborators**
Load pattern report and screening scores as weighted inputs	PatternReport
Apply configurable weighted scoring algorithm	ScreeningRecord
Classify student into Low / Medium / High risk tier	RiskProfile
Update student risk profile in the database	NotifController
Trigger counsellor alert if High threshold is crossed	ScoringInterface
Flag borderline scores for manual counsellor review	AnalyticsController

1.2.3 Class: NotifController

Class: NotifController [*Control*]	
Responsibilities	**Collaborators**
Compose type-specific notification messages (alert, reminder, wellness)	NotificationLog
Deliver in-app notification via UI	NotificationUI
Send email via EmailService; initiate retry on failure	EmailService
Log delivery status and acknowledgement	RiskProfile
Enforce frequency capping to prevent notification fatigue	CounselingSession

1.2.4 Class: AnalyticsController

Class: AnalyticsController [*Control*]	
Responsibilities	**Collaborators**
Trigger weekly and on-demand pattern analysis jobs	StudentAcadData
Load multi-source student data from analytics layer	MLModelInterface
Store pattern analysis results as PatternReport	PatternReport
Flag students with <4 weeks of data for manual review	RiskController
Coordinate dashboard refresh after each analysis cycle	DashboardReport

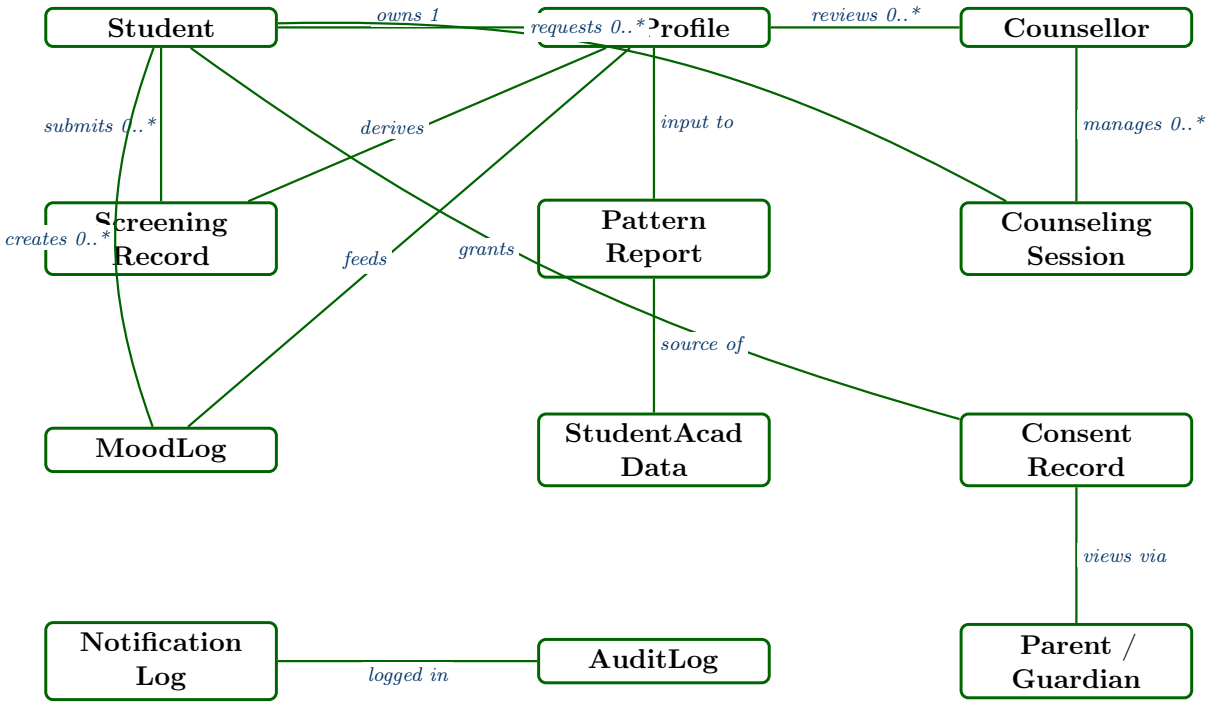
2 IDENTIFYING ASSOCIATIONS

Definition: An **association** is a semantic relationship between two or more classes that implies a structural connection at runtime — one object holds a reference to, creates, or communicates with another object to fulfil use-case behaviour. Associations are identified by scanning CRC card collaborator columns and cross-referencing sequence diagrams from Experiment 04B.

#	Class A	Associatio	Class B	Mult.	Source UC
A-01	Student	*submits*	Screening-Record	1 → 0..*	UC-02
A-02	Student	*creates*	MoodLog	1 → 0..*	UC-03
A-03	Student	*owns*	RiskProfile	1 → 1	UC-04
A-04	Student	*requests*	Counseling-Session	1 → 0..*	UC-05
A-05	Student	*grants consent to*	ConsentRecord	1 → 0..1	UC-07
A-06	Counsellor	*manages*	Counseling-Session	1 → 0..*	UC-14
A-07	Counsellor	*reviews*	RiskProfile	1 → 0..*	UC-13
A-08	Admin	*manages*	User	1 → 0..*	UC-16
A-09	Admin	*configures*	SystemConfig	1 → 1	UC-18
A-10	Admin	*generates*	Dashboard-Report	1 → 0..*	UC-17
A-11	Parent / Guardian	*views via*	ConsentRecord	1 → 0..1	UC-07
A-12	RiskProfile	*derived from*	Screening-Record	1 → 1..*	UC-10
A-13	RiskProfile	*derived from*	PatternReport	1 → 1..*	UC-10
A-14	RiskProfile	*derived from*	MoodLog	1 → 0..*	UC-10

#	Class A	Associatio	Class B	Mult.	Source UC
A-15	PatternReport	*produced from*	StudentAcad-Data	1 → 1..*	UC-09
A-16	NotificationLog	*sent to*	User	0..* → 1	UC-06
A-17	Counseling-Session	*linked to*	Student	1 → 1	UC-05
A-18	Counseling-Session	*linked to*	Counsellor	1 → 1	UC-14
A-19	AuditLog	*records actions of*	User	0..* → 1	UC-29
A-20	StudentAcad-Data	*collected for*	Student	0..* → 1	UC-08
A-21	Dashboard-Report	*aggregates*	RiskProfile	1 → 0..*	UC-12, UC-17
A-22	Dashboard-Report	*aggregates*	Counseling-Session	1 → 0..*	UC-12, UC-17

Association Diagram

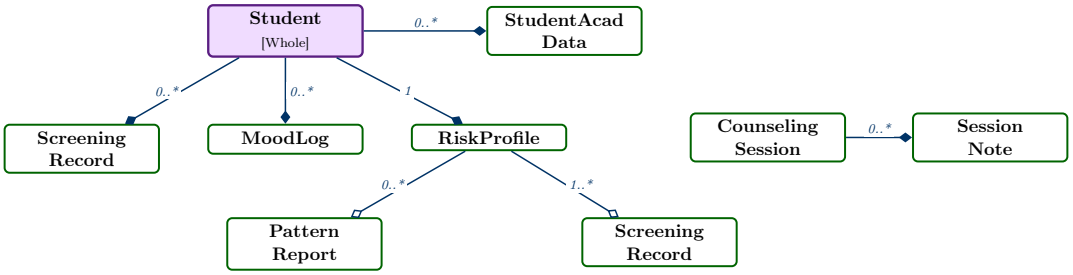


3 IDENTIFYING AGGREGATES

Definition: An **aggregate** (whole–part) relationship exists when one object is composed of or contains a set of other objects as parts. Two subtypes are identified: **Aggregation** (*hollow diamond* — parts can exist independently) and **Composition** (*filled diamond* — parts are exclusively owned and destroyed with the whole). In the UML diagrams below the diamond is placed at the *whole* side.

#	Whole	Type	Part	Mult.	Rationale
AG-01	**Student**	Composition	ScreeningRecord	1 ◇ 0..*	Screening records belong exclusively to one student; deleted if the student account is hard-deleted.
AG-02	**Student**	Composition	MoodLog	1 ◇ 0..*	Daily mood logs are exclusively owned by the student; meaningless without their student owner.
AG-03	**Student**	Composition	RiskProfile	1 ◇ 1	Each student owns exactly one active risk profile; meaningless without its student owner.
AG-04	**Student**	Composition	StudentAcadData	1 ◇ 0..*	Academic data snapshots are exclusively associated with a specific student record.
AG-05	**RiskProfile**	Aggregation	PatternReport	1 ◇ 0..*	Pattern reports contribute to risk computation but remain independently stored for audit.
AG-06	**RiskProfile**	Aggregation	ScreeningRecord	1 ◇ 1..*	Screening records feed risk computation but exist independently for clinical reference.
AG-07	**Counseling-Session**	Composition	SessionNote	1 ◇ 0..*	Session notes are created within and belong exclusively to a counselling session.
AG-08	**Dashboard-Report**	Aggregation	RiskProfile	1 ◇ 0..*	Risk profiles are aggregated into reports but continue to exist as individual records.
AG-09	**Dashboard-Report**	Aggregation	Counseling-Session	1 ◇ 0..*	Session statistics are aggregated but sessions are independent business objects.
AG-	**System**	Composition	AuditLog	1 ◇ 0..*	Audit logs are an

Aggregate Structure Diagram



4 IDENTIFYING ATTRIBUTES

Methodology: Attributes are identified by examining (a) the information each class must *know* to fulfil its CRC responsibilities, (b) the data fields referenced in use-case descriptions and sequence diagrams, and (c) any fields required by the persistence layer. Each attribute is listed with its type, visibility, and derivation notes where applicable.

Class: User

[*Abstract Entity*]

Attribute	Type	Visibility	Notes
userId	String	–	Immutable UUID; primary key
email	String	–	Institutional email; unique; validated format
passwordHash	String	–	Bcrypt hash; never stored in plaintext
role	Enum	–	{Student, Counsellor, Admin, Guardian, ITOfficer}
locked	Boolean	–	True after 5 consecutive failed logins
failedAttempts	Integer	–	Reset to 0 on successful login
createdAt	DateTime	–	Account creation timestamp
lastLoginAt	DateTime	–	Updated on every successful session creation
twoFAEnabled	Boolean	–	Mandatory for Admin; optional for Student

Class: ScreeningRecord

[*Entity*]

Attribute	Type	Visibility	Notes
recordId	String	–	UUID; primary key
studentId	String	–	FK → User.userId
phq9Score	Integer	–	Range 0–27; clinically validated
gad7Score	Integer	–	Range 0–21
pssScore	Integer	–	Range 0–40
riskTier	Enum	–	{Low, Medium, High}; derived at submission
version	Integer	–	Incremented on each re-submission
submittedAt	DateTime	–	UTC timestamp; immutable after save
isComplete	Boolean	–	False if partial-save was used

Class: MoodLog [*Entity*]

Attribute	Type	Visibility	Notes
logId	String	–	UUID; primary key
studentId	String	–	FK → User.userId
moodScore	Integer	–	1 (very low) to 10 (very high)
sleepHours	Float	–	Hours slept last night; range 0–24
stressors	String	–	Free-text; max 500 characters
logDate	Date	–	Date of check-in (not datetime; one per day)
isPartial	Boolean	–	True if submitted with missing optional fields

Class: RiskProfile [*Entity*]

Attribute	Type	Visibility	Notes
profileId	String	–	UUID; primary key
studentId	String	–	FK → User.userId; unique (one profile per student)
compositeScore	Float	–	Weighted aggregate 0.0–100.0; derived
riskTier	Enum	–	{Low, Medium, High}; derived from compositeScore
trendData	JSON	–	Time-series array of (date, score) pairs
classifiedAt	DateTime	–	Timestamp of last risk computation
alertSent	Boolean	–	True if a High-risk alert was dispatched
counsellorId	String	–	FK → User.userId of assigned counsellor

Class: CounselingSession [*Entity*]

Attribute	Type	Visibility	Notes
sessionId	String	–	UUID; primary key
studentId	String	–	FK → User.userId
counsellorId	String	–	FK → User.userId
slotDateTime	DateTime	–	Scheduled date and time
sessionType	Enum	–	{InPerson, Virtual}
status	Enum	–	{Requested, Confirmed, Completed, Cancelled, NoShow}
cancellationNote	String	–	Mandatory if status = Cancelled; max 300 chars
sessionNotes	String	–	Encrypted post-session notes; counsellor-only access
reminderSent	Boolean	–	True after 24-hour automated reminder dispatched
createdAt	DateTime	–	Request creation timestamp

Class: PatternReport [*Entity*]

Attribute	Type	Visibility	Notes
reportId	String	–	UUID; primary key
studentId	String	–	FK → User.userId
anomalies	JSON	–	Array of detected anomaly objects {type, severity, detectedAt}
dataSufficient	Boolean	–	False if <4 weeks of data available
generatedAt	DateTime	–	Timestamp of ML model run
modelVersion	String	–	Version of ML model that produced this report

Class: ConsentRecord [*Entity*]

Attribute	Type	Visibility	Notes
consentId	String	–	UUID; primary key
studentId	String	–	FK → User.userId
guardianId	String	–	FK → User.userId (Guardian role)
granted	Boolean	–	Current consent status
grantedAt	DateTime	–	Last consent grant timestamp
revokedAt	DateTime	–	Null if not yet revoked

Class: StudentAcadData [*Entity*]

Attribute	Type	Visibility	Notes
dataId	String	–	UUID; primary key
studentId	String	–	FK → User.userId
grades	JSON	–	{courseId, score, maxScore, date} array
attendance	JSON	–	{date, present: Boolean} array
engagementScore	Float	–	Derived from LMS activity; normalised 0–1
collectedAt	DateTime	–	SIS API fetch timestamp
syncVersion	Integer	–	Incremental sync counter

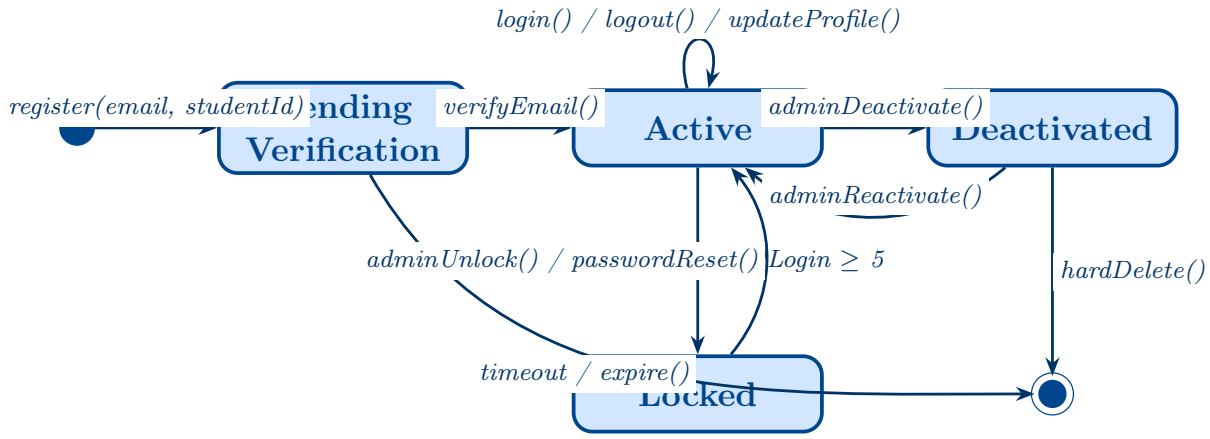
Class: AuditLog [*Entity*]

Attribute	Type	Visibility	Notes
logId	String	–	UUID; primary key
actorId	String	–	FK → User.userId of the acting user
actionType	Enum	–	{Login, DataAccess, ConfigChange, AlertAck, Export, ...}
targetType	String	–	Name of affected entity class
targetId	String	–	UUID of affected entity instance
timestamp	DateTime	–	UTC timestamp; immutable
ipAddress	String	–	Actor's IP at time of action
hashChain	String	–	HMAC of previous log entry; tamper-evidence

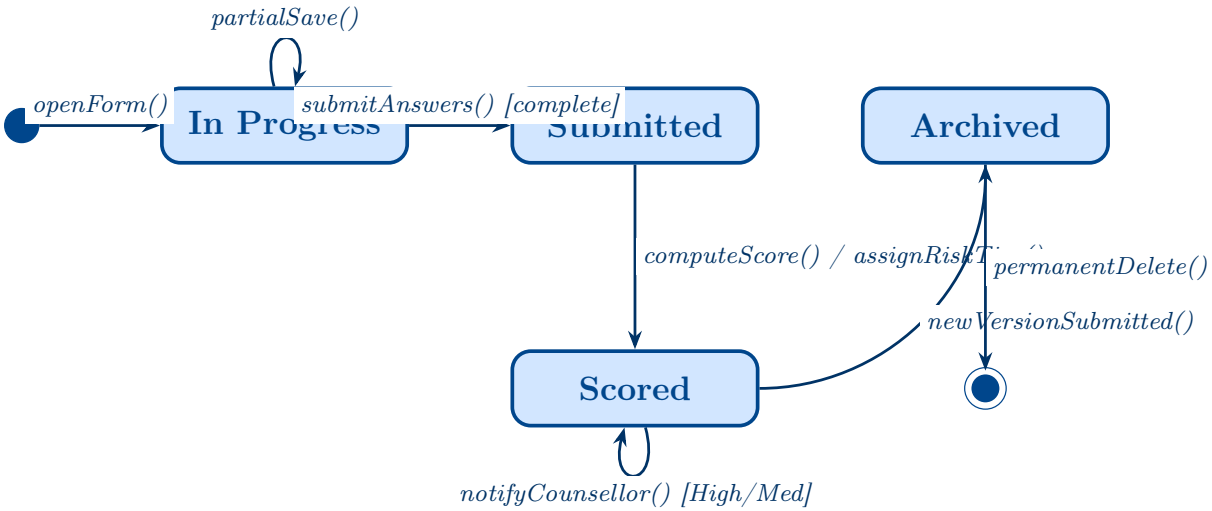
5 MODEL STATE-DEPENDENT BEHAVIOR

Definition: A **State Machine Diagram** models the lifecycle of a single object — the distinct states it can be in, the events that cause it to transition between states, and any actions executed on entry, exit, or during a transition. State machines are drawn for all classes that exhibit non-trivial, event-driven lifecycles.

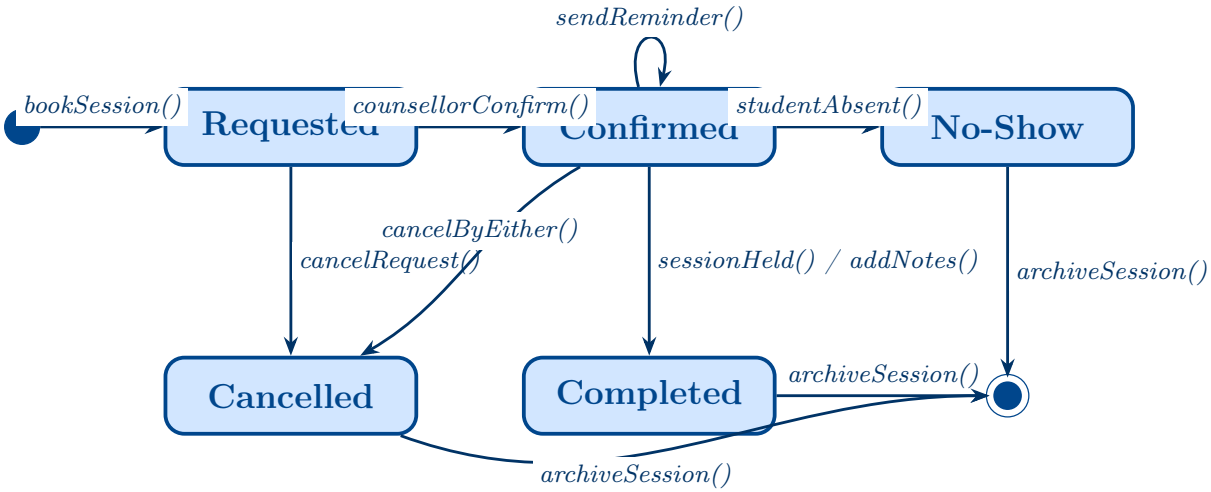
5.1 State Machine: User Account



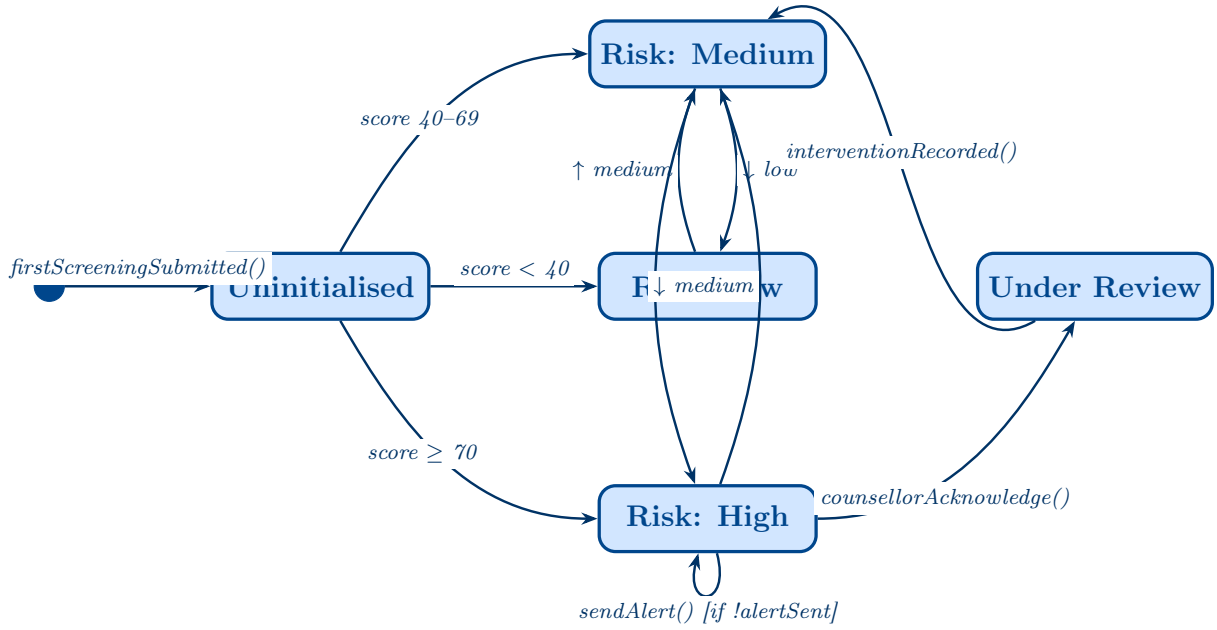
5.2 State Machine: ScreeningRecord



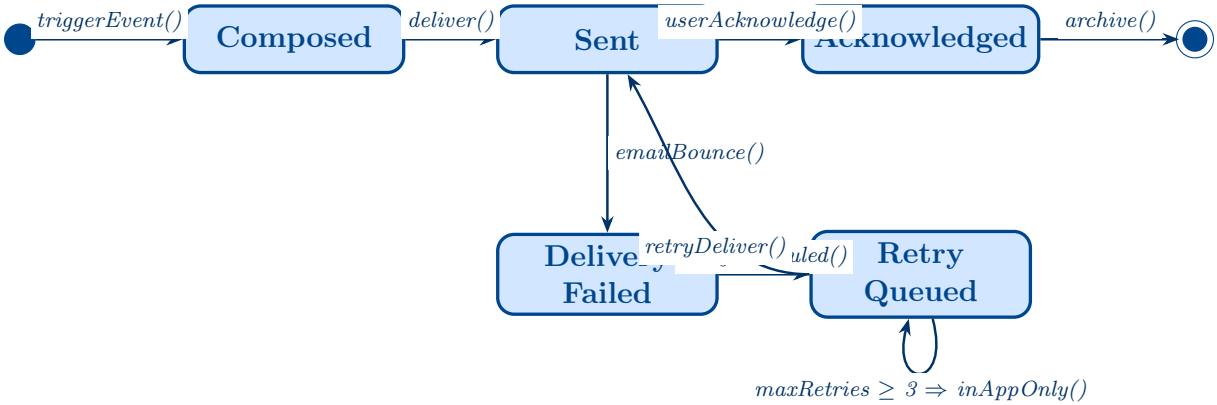
5.3 State Machine: CounselingSession



5.4 State Machine: RiskProfile



5.5 State Machine: NotificationLog

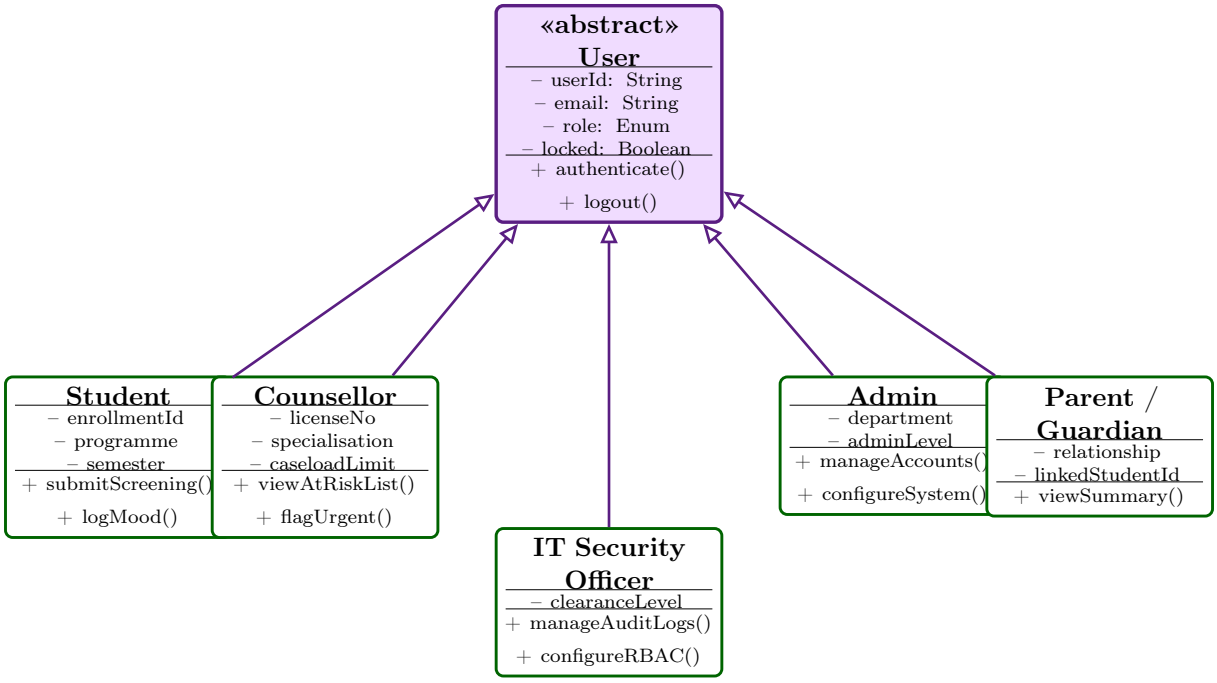


6 MODELING INHERITANCE RELATIONSHIPS

Definition: Generalisation (inheritance) captures *is-a* relationships between classes. A superclass defines shared attributes and operations; each subclass specialises behaviour without repeating common structure. Open triangle arrows point toward the superclass. Abstract classes are denoted with *«abstract»*.

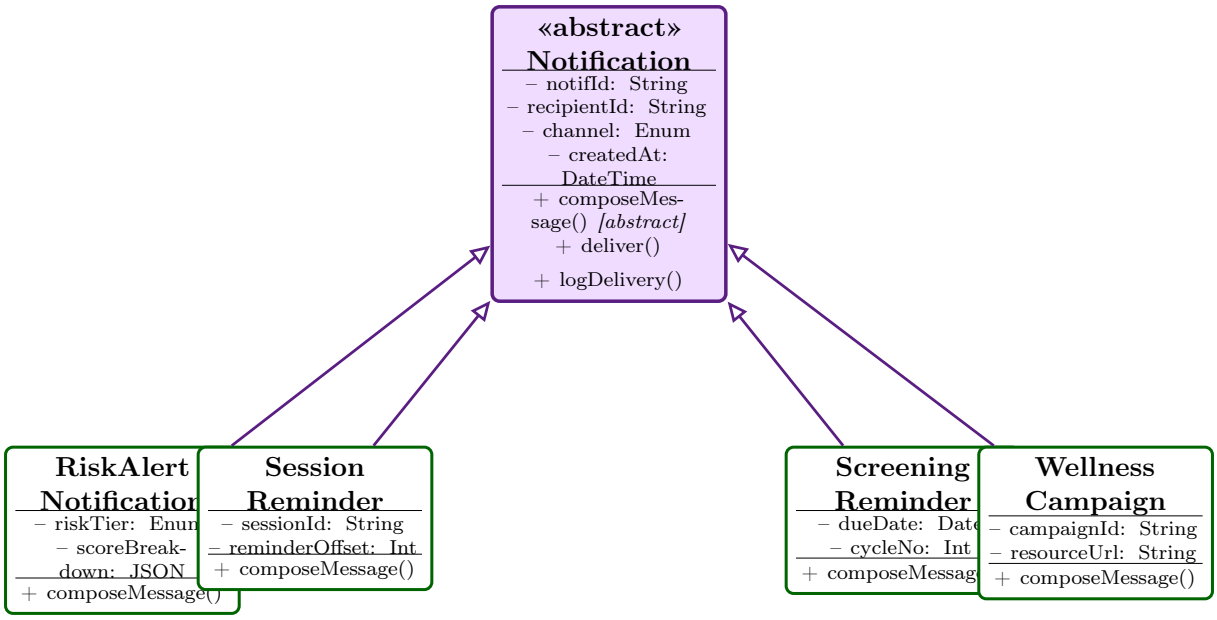
6.1 Hierarchy 1: User Generalisation

This is the primary inheritance hierarchy of the system. All actors in the system are specialisations of the abstract **User** superclass, which holds identity and authentication attributes common to every role.



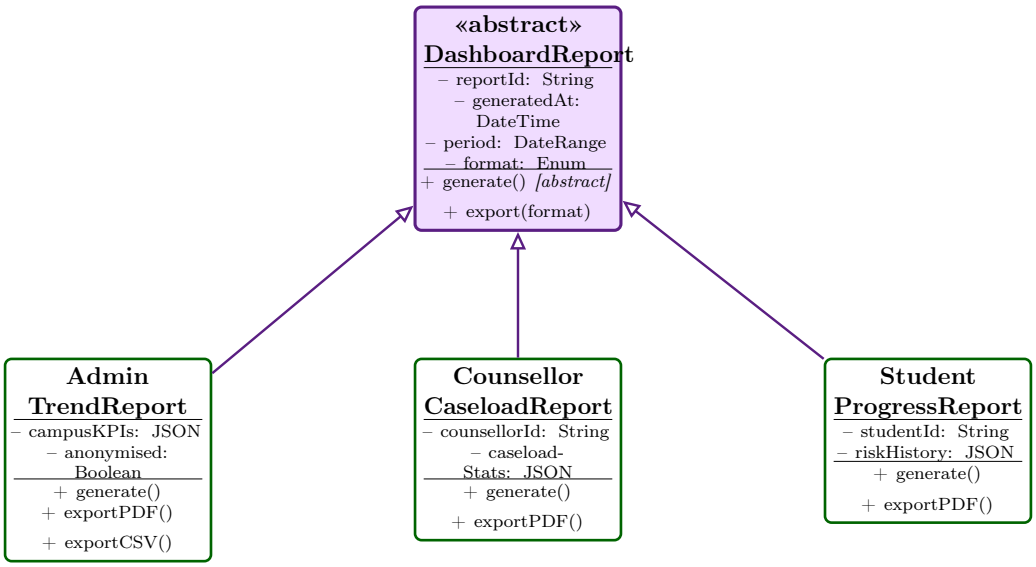
6.2 Hierarchy 2: Notification Generalisation

Notifications in the system are specialised by their purpose and delivery rules. A common abstract **Notification** class captures shared attributes; each subtype overrides `composeMessage()` with purpose-specific content and tone.



6.3 Hierarchy 3: Dashboard Report Generalisation

The **DashboardReport** abstract class is specialised into role-scoped report types. Each subtype differs in its data scope, access restrictions, and export formats.



6.4 Inheritance Summary

#	Superclass	Subclass(es)	Specialisation Rationale
I-01	User *(abstract)*	Student, Counsellor, Admin, Guardian, IT Security Officer	All actors share authentication attributes but differ in role-specific responsibilities, data scope, and dashboards. Generalisation avoids duplication of identity management logic.
I-02	Notification *(abstract)*	RiskAlert-Notification, SessionReminder, ScreeningReminder, WellnessCampaign	All notifications share delivery infrastructure (channel, logging, retry) but each subtype composes a distinct message body and enforces different delivery urgency and frequency rules.
I-03	Dashboard-Report *(abstract)*	AdminTrendReport, CounsellorCaseload-Report, StudentProgress-Report	All reports share a generation pipeline and export mechanism, but differ in data scope, access permissions, and the degree of anonymisation applied at export.

Design Principles Applied

- **Single Responsibility:** Each class in the model is responsible for one cohesive concern — e.g., `RiskProfile` only models risk state; alerting is delegated to `NotifController`.
- **Open / Closed Principle:** The abstract `User` and `Notification` hierarchies allow new roles or notification types to be added by subclassing without modifying existing classes.
- **Dependency Inversion:** Control classes depend on entity abstractions, not concrete database implementations — supporting substitutable persistence layers.
- **Privacy by Design:** Every entity class that holds PII (User, ScreeningRecord, MoodLog, ConsentRecord) has explicit visibility (–) on sensitive attributes, reflecting AES-256 encryption at rest requirements from the problem statement.
- **Tamper-Evident Audit Trail:** The `AuditLog.hashChain` attribute implementing HMAC-linked entries ensures all access and configuration-change events are forensically verifiable.